

HUSSEINI SULEIMAN

2025/FWSD/11478

fwsd2511478@students.icdfa.edu.ng

DATE: 5TH September, 2026

COURSE CODE: SBT-DF202 Lab 4 — Steganography and Hidden Data Analysis

SCENARIO

Your practical context

You are acting as a junior digital forensic trainee. During a basic evidence review, an image file is suspected to contain hidden information. Your task is to understand how steganography works, prepare and preserve a carrier image, hide a controlled evidence note inside the image, extract the concealed file using the correct password, compare file hashes, and test a password-recovery workflow using a controlled wordlist.

All activities must be performed only on the authorized lab image and files created by you within the training environment.

OBJECTIVES

What you must demonstrate

By completing this lab, students will demonstrate their ability to explain the difference between steganography and encryption, describe insertion and substitution-based hiding techniques including least significant bits (LSB), prepare a carrier image and secret evidence file, use Steghide to embed and extract concealed data, calculate and compare cryptographic hashes, perform a controlled password dictionary test using StegSeek, and document all activities in a repeatable forensic report

INSTRUCTOR BRIEFING

Laboratory guidance

Before beginning this lab, review the **Module 8 lessons and study material on Steganography and Hidden Data Analysis**. You must understand the concepts of hidden information, substitution, least significant bits, bitmap images, Steghide, extraction and password recovery before performing the practical tasks.

Required Carrier Image

Download the authorised lab BMP image:

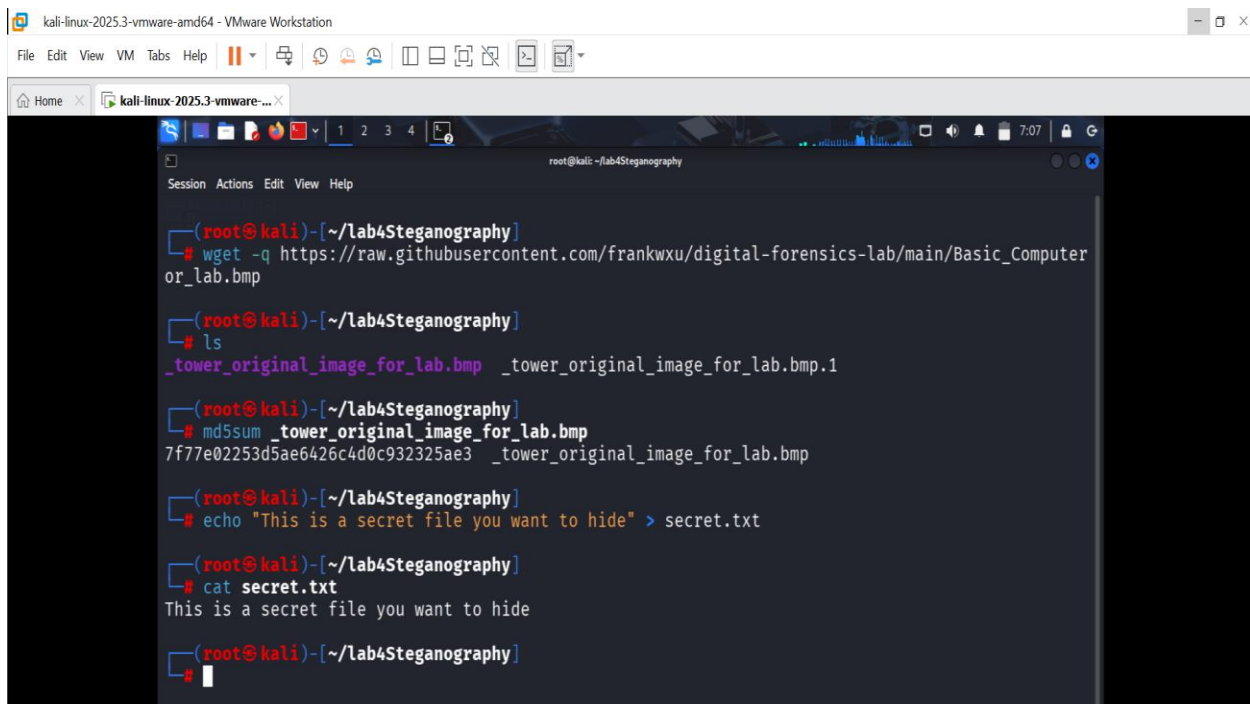
Steganography Lab Carrier Image

If browser download is required, use the link above. The source practical identifies this specific image as the lab carrier.

Part A — Prepare and Preserve the Evidence

Attempt on part

Lab environment preparatory on root@kali using the following for

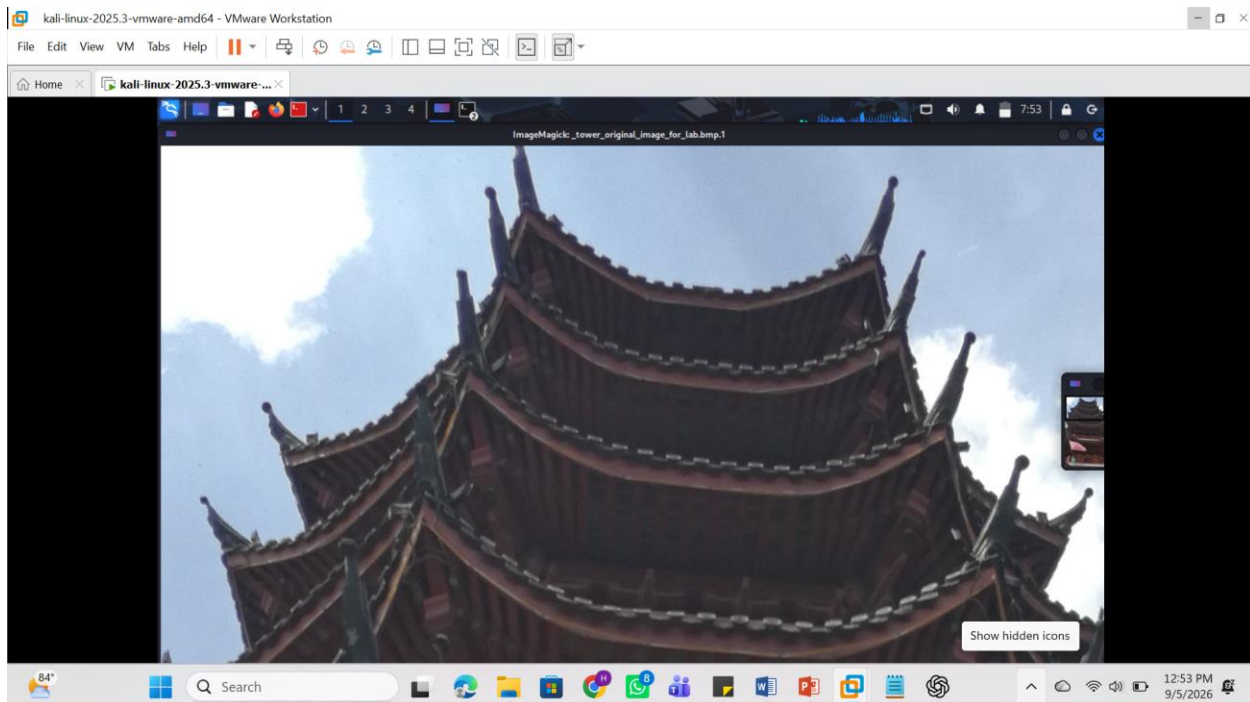
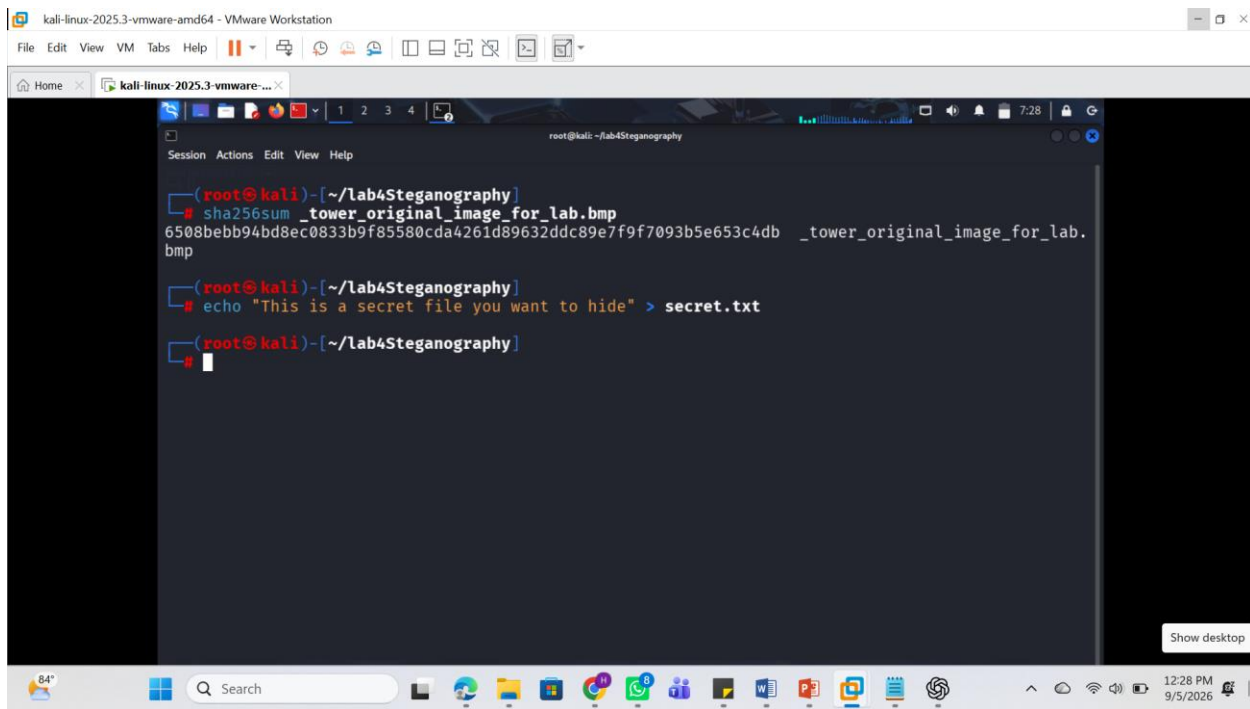


```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-amd64
root@kali: ~/lab4Steganography
Session Actions Edit View Help
(root@kali)-[~/lab4Steganography]
# wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_or_lab.bmp
(root@kali)-[~/lab4Steganography]
# ls
_tower_original_image_for_lab.bmp _tower_original_image_for_lab.bmp.1
(root@kali)-[~/lab4Steganography]
# md5sum _tower_original_image_for_lab.bmp
7f77e02253d5ae6426c4d0c932325ae3 _tower_original_image_for_lab.bmp
(root@kali)-[~/lab4Steganography]
# echo "This is a secret file you want to hide" > secret.txt
(root@kali)-[~/lab4Steganography]
# cat secret.txt
This is a secret file you want to hide
(root@kali)-[~/lab4Steganography]
#
```

Create a dedicated working folder, download the carrier image, confirm its file type and calculate its SHA-256 hash.

Attempt

Creating a dedicated working folder, confirming it's file type and calculating SHA-256 hash.



Preserve the original downloaded image. Do not overwrite it during the embedding exercise.

Part B — Create the Secret Evidence File

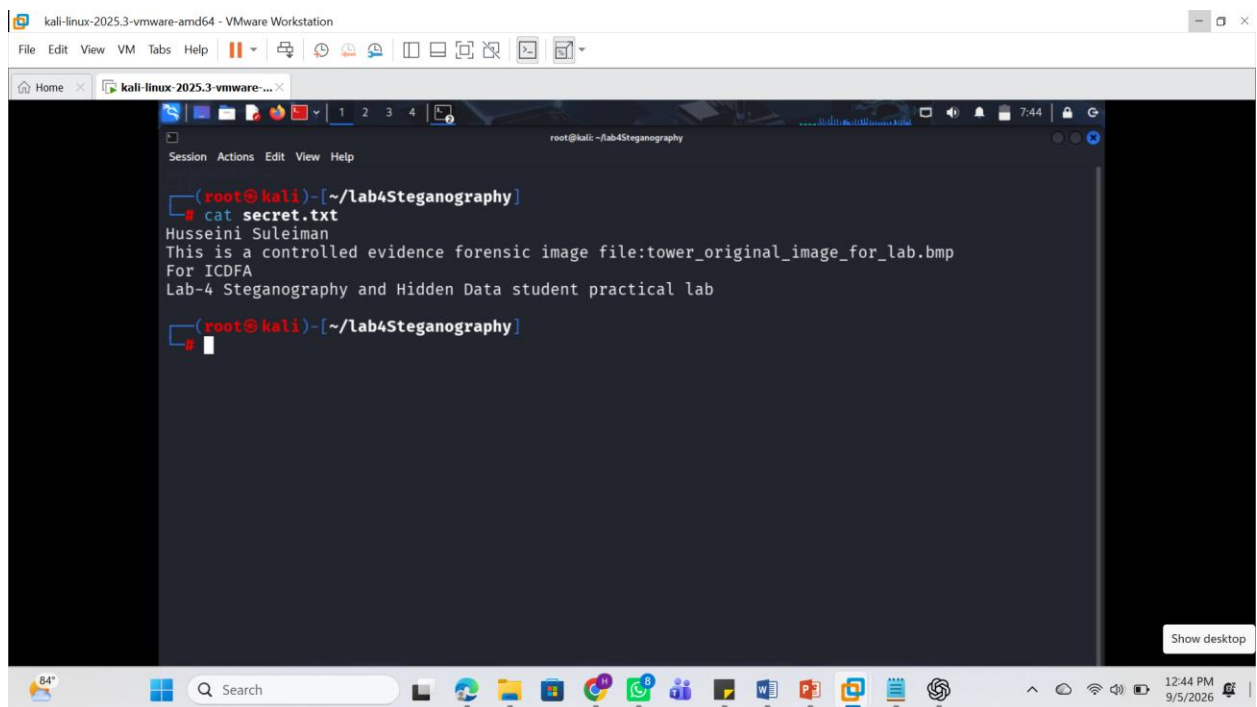
ATTEMPT:

Create a text file named:

secret.txt

Include:

- Your full name
- A short controlled evidence message
- Confirmation that the file was created for the ICDFA lab



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Home kali-linux-2025.3-vmware-amd64
root@kali: ~/lab4Steganography
Session Actions Edit View Help
root@kali)~[~/lab4Steganography]
# cat secret.txt
Husseini Suleiman
This is a controlled evidence forensic image file:tower_original_image_for_lab.bmp
For ICDFA
Lab-4 Steganography and Hidden Data student practical lab
root@kali)~[~/lab4Steganography]
#
```

Calculate and record the SHA-256 hash of the file.

Hash value

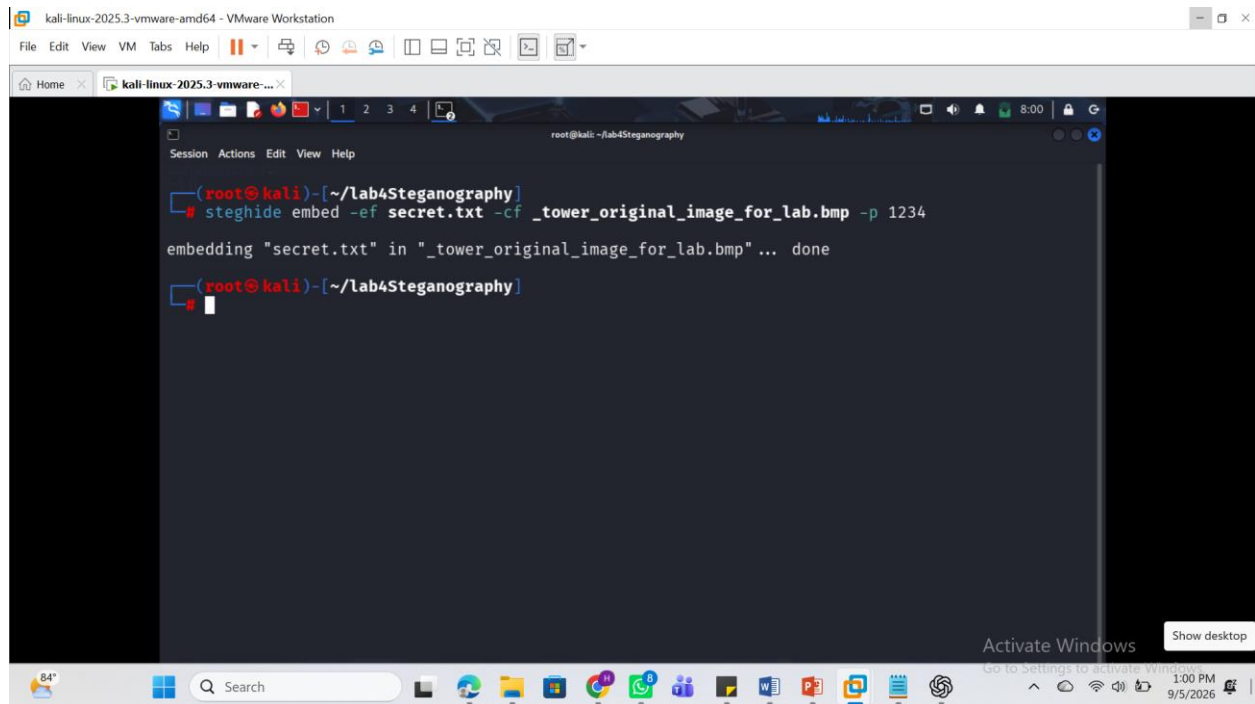
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db

Part C — Embed the Hidden File with Steghide

Use Steghide to embed secret.txt into a **new stego image**.

Using the command: `steghide embed -ef secret.txt -cf _tower_original_image_for_lab.bmp -p 1234`

See output below.



```
root@kali: ~/lab4Steganography
Session Actions Edit View Help
root@kali)~[/lab4Steganography]
# steghide embed -ef secret.txt -cf _tower_original_image_for_lab.bmp -p 1234
embedding "secret.txt" in "_tower_original_image_for_lab.bmp" ... done
root@kali)~[/lab4Steganography]
#
```

For the guided portion of the lab, use the lab password:

1234

The source lab uses this password specifically so the later dictionary-recovery task can be completed.

Example structure:

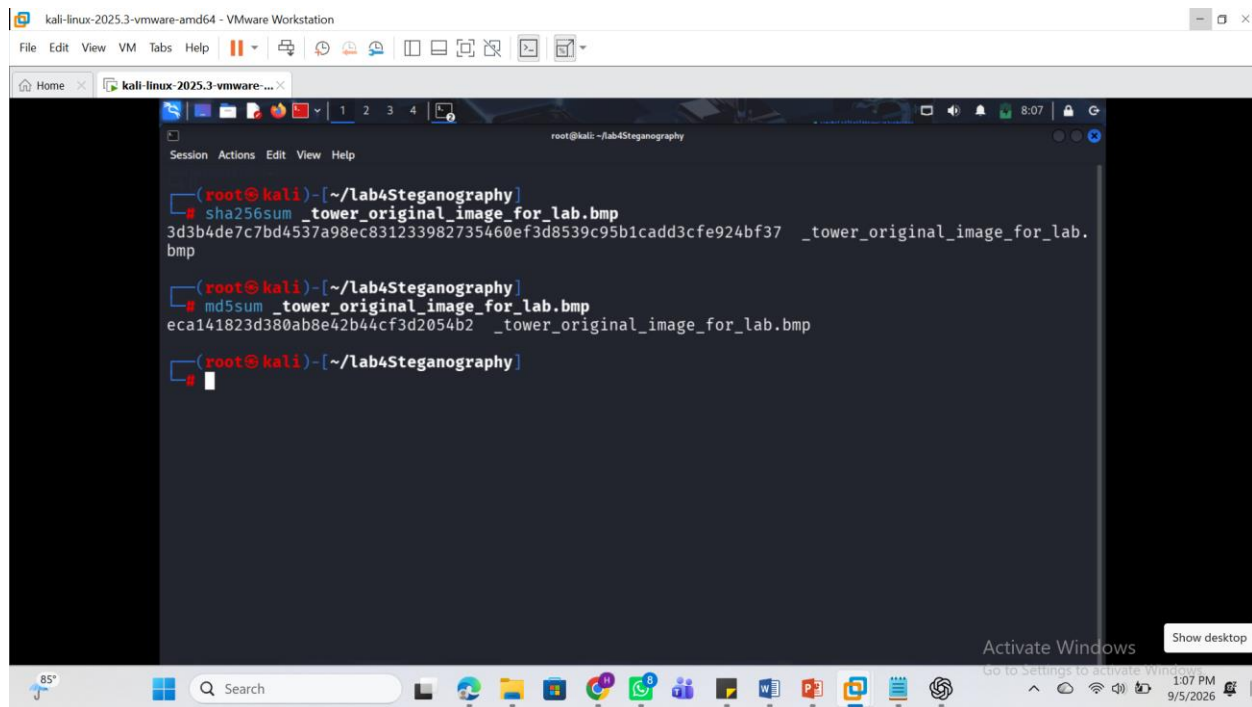
```
steghide embed \ -ef secret.txt \ -cf tower_original_image_for_lab.bmp \ -
sf tower_stego_lab4.bmp \ -p 1234
```

Calculate and record the hash of the resulting stego image.

See the command used and the evidence screenshot

```
(sha256sum _tower_original_image_for_lab.bmp)
```

```
(md5sum _tower_original_image_for_lab.bmp)
```



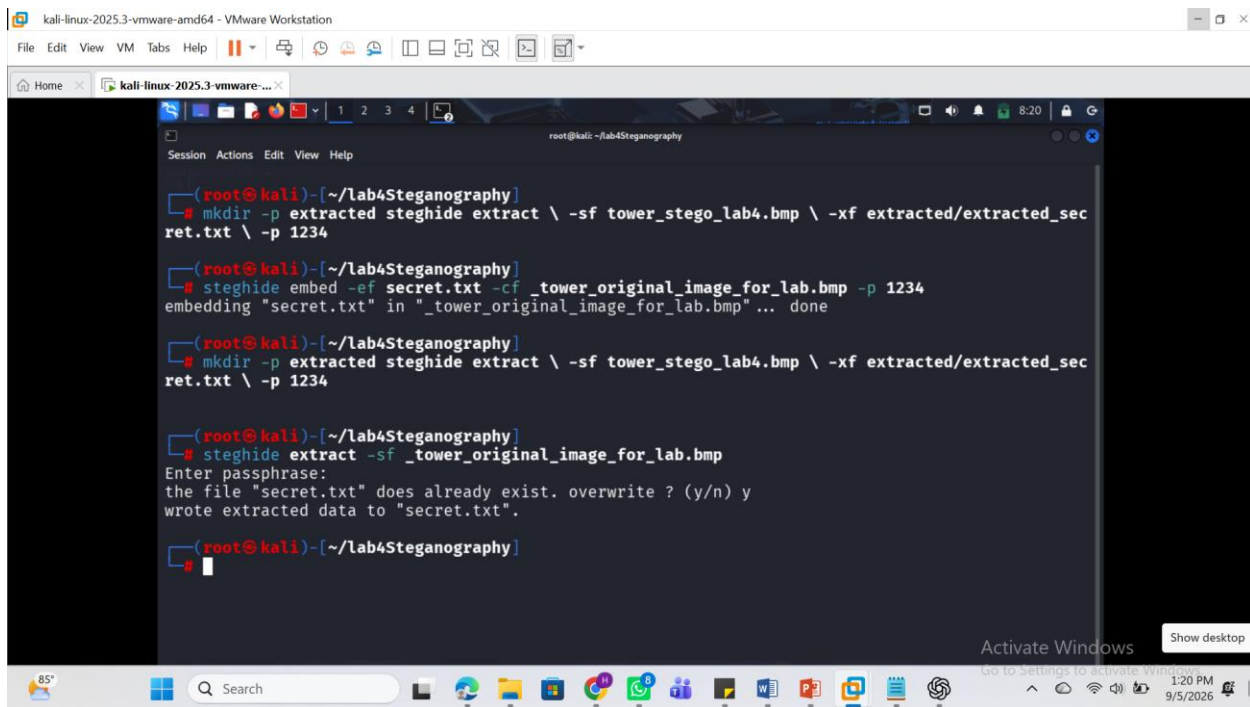
Part D — Extract and Verify the Hidden Evidence

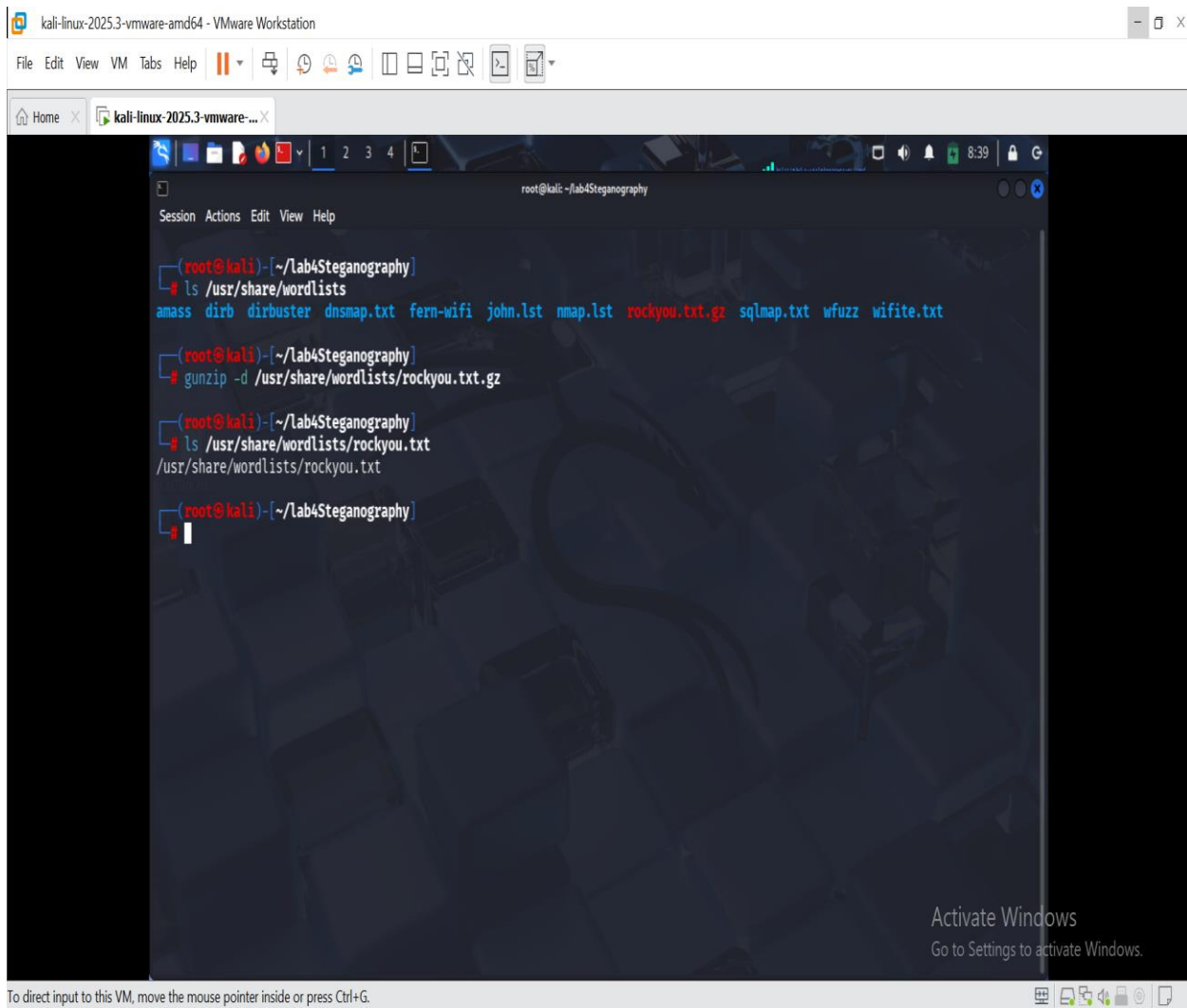
Extract the concealed file using the correct password.

Example:

```
mkdir -p extracted steghide extract \-sf tower_stego_lab4.bmp \-  
xf extracted/extracted_secret.txt \-p 1234
```

Compare the original and extracted files using diff and/or SHA-256 hashes. The recovered file should match the original if extraction is successful.





Part E — Compare Original and Stego Images

Compare:

- File sizes: -rw-rw-r-- 1 root root 152 Sep 5 08:42 secret.txt

SHA-256 hashes:

a7503697ccd3f37ae8336ca3d875b39e9c104fe4786608c42ccdad9fa37e782c

- Readable strings where applicable: command used xxd
_tower_original_image_for_lab.bmp

Explain why two images may look visually similar while their underlying digital content and hashes are different.

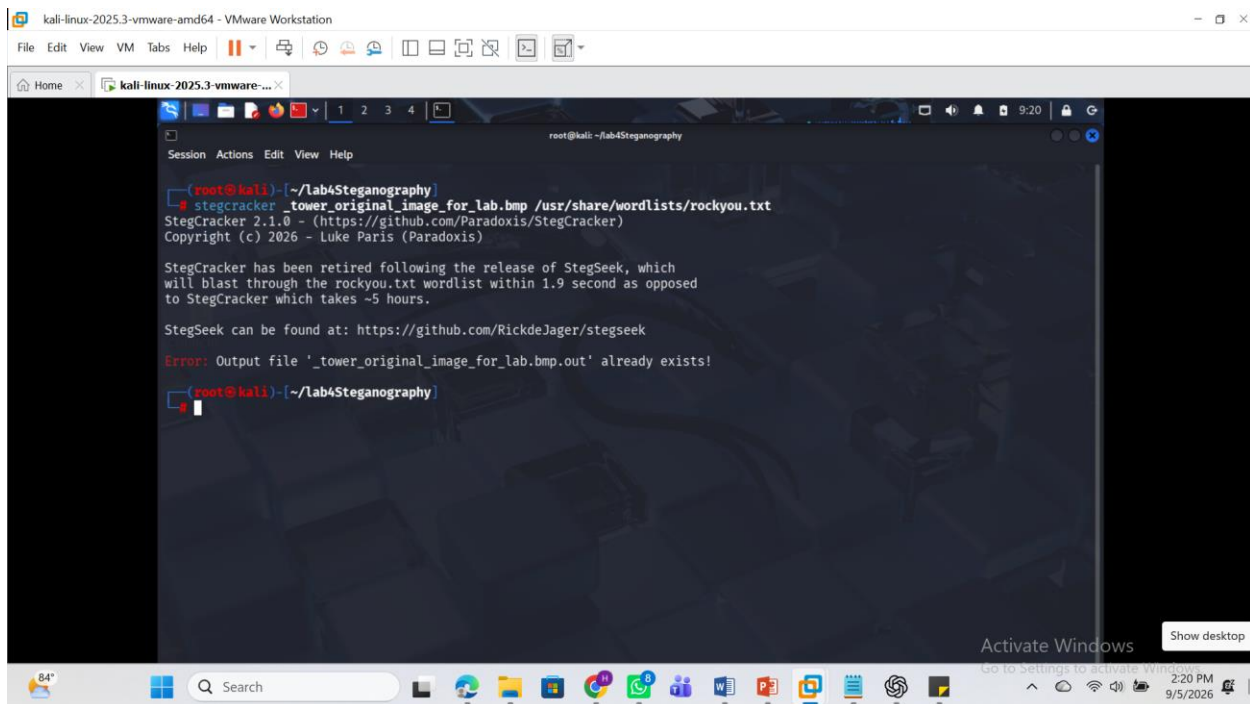
Part F — Controlled Password Dictionary Test

Create a small lab wordlist that contains the known password 1234.

Use StegSeek against the stego image:

```
stegseek tower_stego_lab4.bmp lab4_wordlist.txt
```

Document the result.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Home kali-linux-2025.3-vmware-amd64
root@kali:~/lab4Steganography
stegcracker tower_original_image_for_lab.bmp /usr/share/wordlists/rockyou.txt
StegCracker 2.1.0 - (https://github.com/Paradoxis/StegCracker)
Copyright (c) 2026 - Luke Paris (Paradoxis)

StegCracker has been retired following the release of StegSeek, which
will blast through the rockyou.txt wordlist within 1.9 second as opposed
to StegCracker which takes ~5 hours.

StegSeek can be found at: https://github.com/RickdeJager/stegseek
Error: Output file '_tower_original_image_for_lab.bmp.out' already exists!
root@kali:~/lab4Steganography
```

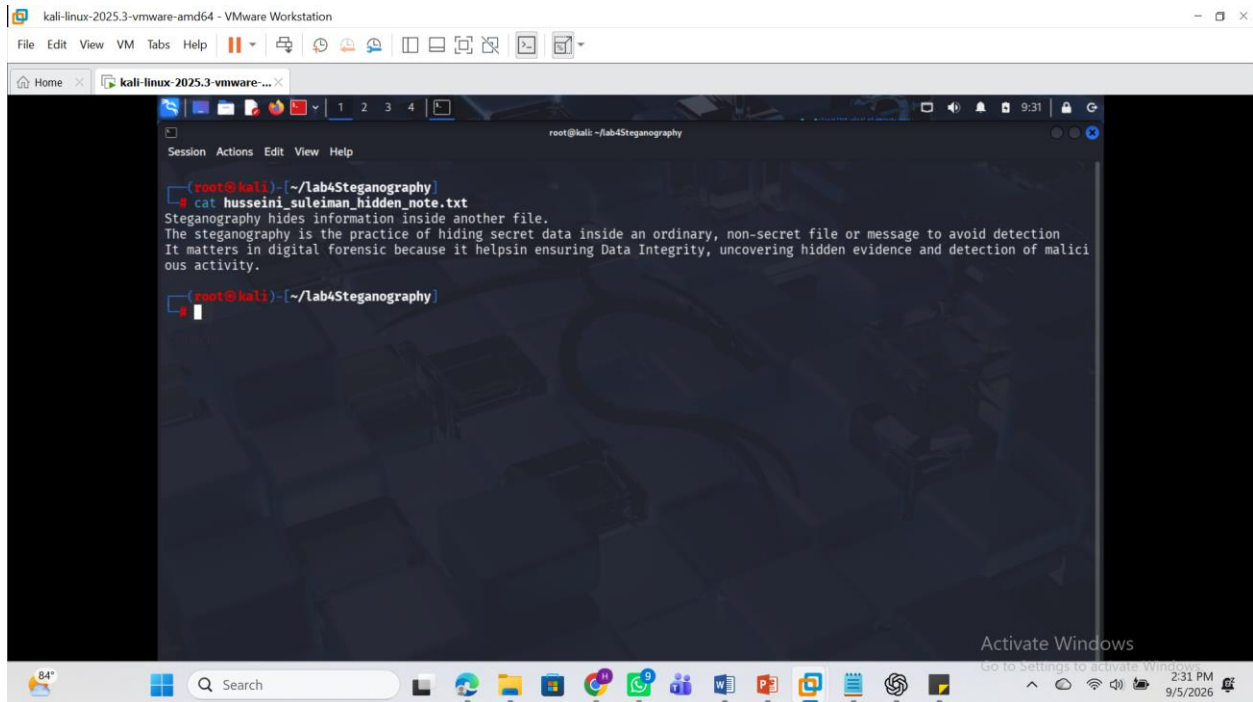
If StegSeek is unavailable or fails in your environment, document the error clearly and complete the Steghide embedding/extraction workflow successfully instead.

Part G — Independent Student Task

Create your own hidden evidence file:

```
firstname_lastname_hidden_note.txt
```

Write a short explanation of what steganography is and why it matters in digital forensics.

A screenshot of a Kali Linux terminal window running inside a VMware Workstation. The terminal shows a root user at a kali machine in the directory ~/Lab4Steganography. The user has executed the command 'cat husseini_suleiman_hidden_note.txt'. The output of the command is a text block explaining steganography: 'Steganography hides information inside another file. The steganography is the practice of hiding secret data inside an ordinary, non-secret file or message to avoid detection. It matters in digital forensic because it helps in ensuring Data Integrity, uncovering hidden evidence and detection of malicious activity.' The terminal window has a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The VMware Workstation window title is 'kali-linux-2025.3-vmware-amd64 - VMware Workstation'. The Windows taskbar at the bottom shows the time as 2:31 PM on 9/5/2026 and includes an 'Activate Windows' watermark.

Create your own hidden evidence file:

firstname_lastname_hidden_note.txt

Write a short explanation of what steganography is and why it matters in digital forensics.

Then:

1. Embed it into the carrier image.
2. Use a password of your choice.
3. Extract it again.
4. Verify integrity using hashes.
5. Create a small wordlist containing your chosen password.
6. Test the recovery workflow.
7. Document every step with screenshots.